Experiment 6 B: Taxi Problem with Q-Learning

Aim: Solve the taxi problem with reinforcement learning: the agent picks up a passenger at one and drops them at their destination.

In [1]:
```
# A52 Dipanshu Ambilkar
import numpy as np
```

In [2]:
```
ROWS, COLS = 5, 5
CORNERS = {"R": (0, 0), "G": (0, 4), "Y": (4, 0), "B": (4, 3)}
ACTIONS = ["up", "down", "left", "right", "pickup", "dropoff"]
print("Grid:", ROWS, "x", COLS, "| actions:", ACTIONS)
```

```
Grid: 5 x 5 | actions: ['up', 'down', 'left', 'right', 'pickup', 'dropoff']
```

In [3]:
```
def step(state, action):
    taxi, passenger, dest = state
    r, c = taxi
    reward, done = -1, False
    if action == "up":
        r = max(0, r - 1)
    elif action == "down":
        r = min(ROWS - 1, r + 1)
    elif action == "left":
        c = max(0, c - 1)
    elif action == "right":
        c = min(COLS - 1, c + 1)
    elif action == "pickup":
        if passenger != "in_taxi" and (r, c) == CORNERS[passenger]:
            passenger = "in_taxi"
        else:
            reward = -10
    elif action == "dropoff":
        if passenger == "in_taxi" and (r, c) == CORNERS[dest]:
            reward, done = 20, True
            passenger = "done"
        else:
            reward = -10
    return ((r, c), passenger, dest), reward, done

print("Test: pickup at the R corner gives", step(((0, 0), "R", "G"), "pic
```

```
Test: pickup at the R corner gives (((0, 0), 'in_taxi', 'G'), -1, False)
```

In [4]:
```
def new_episode(rng):
    corners = list(CORNERS)
    start = corners[rng.integers(4)]
    dest = corners[rng.integers(4)]
    while dest == start:
        dest = corners[rng.integers(4)]
    taxi = (rng.integers(ROWS), rng.integers(COLS))
    return (taxi, start, dest)

print("Sample episode start:", new_episode(np.random.default_rng(1)))
```

```
Sample episode start: ((np.int64(3), np.int64(4)), 'G', 'Y')
```

In [5]:
```
rng = np.random.default_rng(7)
Q = {}

def q_values(state):
    if state not in Q:
        Q[state] = np.zeros(len(ACTIONS))
    return Q[state]

print("Q table starts empty, states so far:", len(Q))
```

```
Q table starts empty, states so far: 0
```

In [6]:
```
alpha, gamma, epsilon = 0.1, 0.99, 0.1
print("alpha:", alpha, "| gamma:", gamma, "| epsilon:", epsilon)
```

```
alpha: 0.1 | gamma: 0.99 | epsilon: 0.1
```

In [7]:
```
for ep in range(2000):
    state = new_episode(rng)
    done = False
    while not done:
        if rng.random() < epsilon:
            a = rng.integers(len(ACTIONS))
        else:
            a = int(np.argmax(q_values(state)))
        next_state, reward, done = step(state, ACTIONS[a])
        best_next = np.max(q_values(next_state))
        q_values(state)[a] += alpha * (reward + gamma * best_next - q_val
        state = next_state
    if (ep + 1) % 500 == 0:
        print(f"Episode {ep + 1} done")
print("States learned:", len(Q))
```

```
Episode 500 done
Episode 1000 done
Episode 1500 done
Episode 2000 done
States learned: 404
```

In [8]:
```
state = new_episode(rng)
print("Test episode start state:", state)
```

```
Test episode start state: ((np.int64(2), np.int64(1)), 'Y', 'G')
```

In [9]:
```
done, total, moves_made = False, 0, 0
while not done and moves_made < 100:
    a = int(np.argmax(q_values(state)))
    state, reward, done = step(state, ACTIONS[a])
    total += reward
    moves_made += 1
print("Test episode: total reward =", total, "| moves =", moves_made)
```

```
Test episode: total reward = 8 | moves = 13
```

In [10]:
```
print("The taxi learned to pick up and drop off correctly."
      if total > 0 else "Still learning - run more episodes.")
```

```
The taxi learned to pick up and drop off correctly.
```